Market-Basket Analysis Project
Algorithms for Massive Data

Marco Catoia (67031A)
Master Degree in Data Science for Economics
Università degli Studi di Milano

13 June 2026

Contents

1 Introduction

This project aims at implementing two of the most famous algorithms used to solve the problem of finding frequent itemsets, which consists of discovering sets of items that can be pairs, triples, quadruples, and so on that appear "frequently" in a dataset. In this context the concept of frequent means that this set should appear in the data a number of times that exceeds or is at least equal to a predetermined support threshold s.

These kind of problems are modeled using the so called Market-Basket Model, where a basket is a set composed of items. In our specific task, we are required to find sets of actors that appeared frequently together, therefore each film represents a basket, while each actor represents an item of a basket.

The two algorithms considered are A-Priori and one of its extensions, that is the PCY (Park-Chen-Yu) algorithm. For both the algorithms, frequent couples of actors have been found. Their implementation has been carried out taking as a reference point the theoretical framework explained in class and available in Chapter 6 of [1, Cap. 6, available online at `http://infolab.stanford.edu/~ullman/mmds/ch6.pdf`].

2 Data Acquisition, Description and Preprocessing

The dataset used for this project is the IMDB Movies Dataset, containing the top 1'000 Movies by IMDB Rating. Data are available at the following link: IMDB Movies Dataset, and they have been imported using the Kaggle API.
As mentioned, the dataset contains 1'000 rows, each one representing a movie, and 16 columns, containing information about the title, the year of release, the duration, the genre, the stars and director of the movie and so on. For the purpose of this project only 4 columns out of 16 were considered, which are the columns containing the names of the actors playing a role for the respective movie. Moreover, the presence of missing values for these specific columns has been checked, and since no null value has been found, each basket will be composed of exactly 4 actors. Then, since the dimensions of the dataset are not prohibitive, all the data have been used for the execution of the algorithms.

Before running the algorithms, data have been organized and preprocessed to make them compatible with the theoretical models and to increase computational efficiency. As a first step, instead of encoding data multiple times during the different steps of the algorithms, each actor has been mapped in advance to a unique integer identifier. This encoding has been done by taking a set containing the unique actor names and sorting them in alphabetical order. Then a dictionary having as key the name of the actor and as value the corresponding index of the actor in the set has been created. Of course, also an opposite dic-

tionary has been made, in order then to decode the results of the two algorithms.

After this step, each movie, that is, a basket (a set in Python) containing the IDs of the four actors that played in that movie, has been collected inside a list.

3 A-Priori Algorithm

The first algorithm implemented is A-Priori. The main property that makes A-Priori and its extensions this effective is the property of monotonicity. This property implies that if an itemset is frequent, then also its subsets are frequent, and this allows not to consider a consistent amount of possible candidates.

The algorithm has been structured in passes, each of them has been implemented with a function, and, in the end, a final function that executes all of them has been added. The between passes task has been incorporated in the definition of the precedent pass, instead of being defined separately. With this structure each of the pass function returns a dictionary containing the corresponding frequent itemsets, and this output is then used by the following function. In this specific case, the algorithm implemented is able to compute the number of frequent singletons and couples of actors. Therefore, the passes considered are two and they have been implemented with the following approach:

1. Pass 1: With the first pass, the algorithm goes through the entire dataset and counts the number of appearances of each actor inside a film. After that, the actors that do not reach the support threshold are filtered out, and the ones that are frequent are saved inside a dictionary L1, containing then all the frequent singletons, saved as key value pairs, with the ID as key, and the count as value.

2. Pass 2: In the second pass the algorithm goes again through the whole set of baskets and, for each one of them, filters the actors inside that basket that are present in L1, and computes all possible couples of frequent actors of that basket. After that their occurences are counted. In this implementation, the pairs generated are counted inside the loop during the scan of each basket, so that we avoid going through the whole dataset more than once. Moreover, the pairs of IDs of actors and their respective counts are saved using the triples method, so that we avoid to allocate memory for all possible pairs. In the end with the between pass we filter out infrequent pairs and we create L2.

Once the algorithm has been implemented, the choice to make was related to the support threshold. In the code, it is expressed in relative terms and then it is converted in absolute ones, according to the number of baskets. In this case, the choice of the support threshold was made after different tries. Initially, it has been fixed around 1%, but since the results provided were almost null, it has been progressively decreased in order to find a good and interpretable

compromise. In the end, the final choice was to fix it at 0.3%, which in absolute terms means that the actors should have played in at least 3 films together. This threshold not only provides some interpretable outcomes but also makes sense from the domain point of view. Since we are working with the top 1'000 IMDB Movies, a set of actors can be considered frequent even with a threshold that is this low because it is sufficiently difficult to appear together in at least three films considering only the highest rated ones.

4 PCY Algorithm

The PCY (Park-Chen-Yu) is an extension of the A-Priori algorithm, that is very useful to deal with larger datasets. The intuition behind this is that it exploits the main memory available during the first pass by adding an hash table that works as a filter to discard in advance couples that are for sure infrequent. In fact during the first pass, instead of only computing occurrences of singletons, the algorithm hashes each pair of items of a basket into a fixed number of buckets, and each time that pair is encountered we increase the count of the respective bucket. Then all the bucket counters will be converted into a bitmap, that assigns a 0 to bucket whose count is less than the support threshold, and 1 to the others. In this way we can discard many pairs that we know for sure will be infrequent. Of course, for this implementation we are forced to use the triple method. The algorithm has been then structured as follows:

1. Pass 1: In the first pass the algorithm computes occurrences of singular items, providing then only the frequent ones using the support threshold. In addition, it generates all possible pairs inside a basket and hashes each one to a bucket. Finally, a hash table is built, containing the count for each bucket.

2. Bitmap Construction: In this stage, the hash table is used to create a bitmap. In the bitmap, a 1 is assigned to frequent buckets, and a 0 to infrequent ones. This structure is then used to filter out pairs that are not useful to us.

3. Pass 2: In the second pass we find the candidate pairs using L1, but in addition we re-hash these pairs and we check if the associated counter has a value 1 or 0 in the bitmap: if it is 0 it is discarded, if it is 1 it is retained. Then these pairs are counted and the ones below the threshold are discarded. In this way, we obtain the frequent couples of actors.

In the PCY implementation a choice that has to be made is the one relative to the number of buckets to consider in the hashing phase. Tipically, the number of buckets should be fixed to fill the main memory that remains available in the first pass, so that the filtering is very effective. In this specific case, since the memory constraint is not a binding constraint, the default value of the number of buckets has been fixed at 10'000. This because, considering that each basket

contains 4 actors and that the possible combination of actors of a basket are 6, the maximum number of different couples we could observe would be 6'000. So, by fixing a higher number of buckets, we are hopefully achieving an effective filtering.

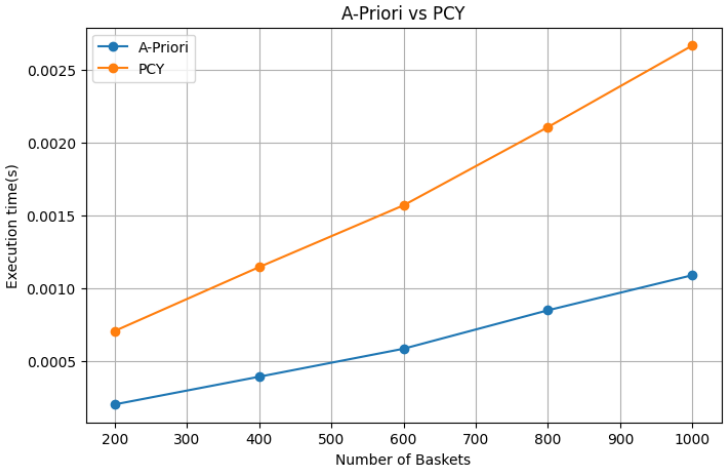
5 Scalability of the Algorithms

To evaluate how the implemented solutions scale as the data size grows, the algorithms were run on different subsamples of the dataset of increasing dimensions, till reaching the complete one, in order to evaluate the time needed to run them. In particular samples composed of 20%, 40%, 60%, 80% and the final dataset with 100% of data were considered. In order to get a more consistent measurement of the execution time, and since the dataset was sufficiently small to allow this kind of approach, an average over 25 runs of the algorithms was taken. Therefore, results will change over different runs, but the variation will hopefully be lower considering this average. As can be seen in the plot and in the table that resumes the times reported below, both algorithms provided an almost linear execution time.

Table 1: Comparison of execution time of A-Priori and PCY

Dataset Size	**A-Priori**	**PCY**
20%	0.0002 s	0.0007 s
40%	0.0004 s	0.0011 s
60%	0.0006 s	0.0016 s
80%	0.0009 s	0.0021 s
100%	0.0011 s	0.0027 s

Figure 1: Plot of the execution time of A-Priori and PCY



Since execution time seems to scale in an almost linear way, we can say that we have an almost linear time complexity. This is a result we could expect, because of the structure of the two algorithms. In fact, in each pass, the algorithms read the whole set of baskets one time and, since the number of pass is fixed, the computational cost grows almost linearly as the number of baskets increases. This result is then optimal for our algorithms, since we cannot avoid to pass through the whole dataset because we need to count occurrences. Therefore, this result suggests that both A-Priori and PCY scale up well with data size.

The plot shows also another interesting and counterintuitive result, that is, the average time for PCY in order to be executed is greater than the time required for the execution of A-Priori. This is probably due to the fact that the analyzed dataset is of small dimensions. In fact, because frequent pairs are already not too much, the time needed for the hashing of the pairs is higher than the time saved due to the filtering of infrequent couples. For this reason, the execution time of PCY is higher. Probably, with a dataset of a higher size, the advantages of PCY can become more evident and its implementation could be more meaningful.

6 Experiment Description and Results

In this final section, we explore then the results of the experiments. As mentioned before, the two algorithms were run on the entire dataset of 1'000 movies. The relative support threshold has been fixed at 0.3% (0.003), which implies that the support threshold is equal to 3. For what concerns instead PCY, the number of buckets was fixed at 10'000, as explained earlier.

As a first step, to verify correctness of the implementation of the two algorithms, the frequent itemsets of both have been compared and the two provided the same exact results for singletons and pairs. The number of single frequent actors found by the algorithms is 271, while the number of frequent couples is instead 25. The table below summarizes the frequent pairs:

Table 2: Frequent couples and their support

Actor 1	**Actor 2**	**Support**
Daniel Radcliffe	Rupert Grint	6
Daniel Radcliffe	Emma Watson	5
Emma Watson	Rupert Grint	5
Tim Allen	Tom Hanks	4
Joe Pesci	Robert De Niro	4
Al Pacino	Diane Keaton	3
Christian Bale	Michael Caine	3
Al Pacino	Robert De Niro	3
Elijah Wood	Ian McKellen	3
Elijah Wood	Orlando Bloom	3
Ian McKellen	Orlando Bloom	3
Carrie Fisher	Harrison Ford	3
Carrie Fisher	Mark Hamill	3
Harrison Ford	Mark Hamill	3
Takashi Shimura	Toshirô Mifune	3
Chris Evans	Joe Russo	3
Chris Evans	Robert Downey Jr.	3
Joe Russo	Robert Downey Jr.	3
Mark Ruffalo	Robert Downey Jr.	3
Tatsuya Nakadai	Toshirô Mifune	3
Ethan Hawke	Julie Delpy	3
Chris Evans	Scarlett Johansson	3
Diane Keaton	Woody Allen	3
Humphrey Bogart	Lauren Bacall	3
Ethan Coen	John Turturro	3

As we can immediately see, the couple with highest appearances is composed of Daniel Radcliffe and Rupert Grint, followed by Daniel Radcliffe and Emma Watson, which are the actors that played a major role in Harry Potter movies. After them, we find Tim Allen and Tom Hanks, probably because of the Toy Story franchise, and Joe pesci and Robert de Niro, probably linked by their partnership with Martin Scorsese. In addition we can recognize some other groups of actors that played together in different sagas, like for example Christian Bale and Michael Caine in the The Dark Knight, Al Pacino and Diane Keaton in the Godfather, and all the middel block composed by Elijah Wood, Ian McKellen and Orlando Bloom, which is related to The Lord of the Rings,

and Carrie Fisher, Harrison Ford, and Mark Hamill, related to Star Wars.

To conclude the analysis, this application demonstrates that the concept of Market-Baket-Analysis can be also applied beyond its classical context and can be used as an effective tool to find meaningful associations between items. In this specific case, the algorithms succeeded in identifying well known couples of actors linked by sagas. However, because of the limited size od the dataset, the results do not show particular actors partnerships that go beyond a particular franchise, besides some exceptions. For this reason, an interesting idea to extend this work could be to execute these algorithms on a larger version of the dataset, that maybe would allow us to enrich our results. Moreover, running them on a bigger dataset, could allow to better see the advantages that PCY can bring with respect to A-Priori.

References

[1] Anand Rajaraman, Jure Leskovec and Jeffrey D. Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2019, `http://www.mmds.org/#ver30`.

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.